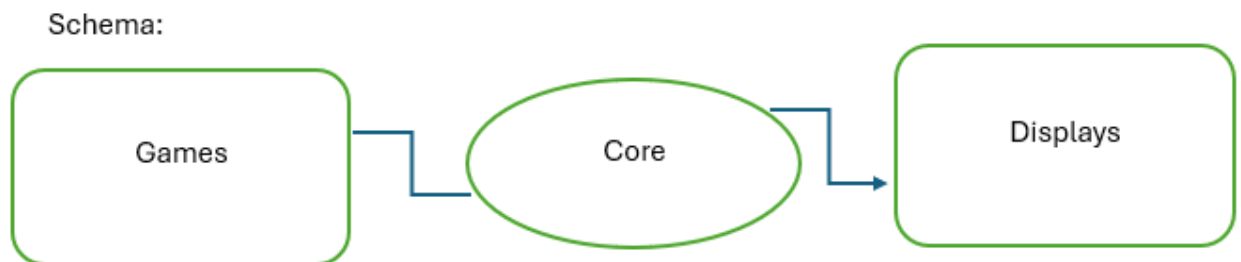


Welcome to our Arcade documentation

Definition: Arcade is a c++ project made in second year at Epitech. The goal of this project is to create our own arcade game with some special features.

This project contains three parts like the part of the games, the graphics librairies, and the core.

- **Games:** This part contains our games (snake, nibbler) that should be displayed in the screen
- **Graphics:** this part contains the librairies (ncurses, sfml and sdl2) in what the games would be displayed.
- **Core:** this part is like the orchester. it is the part that ties the other two together, it is the manager of the dynamic libraries and the one who sends the information from one part to the other.



✓ Core:

- **General:**

The core is the system that allows you to launch the arcade program and connect the different libraries in real time.

- **Keys Management:**

When the events go through the Core in order to be transmitted from the graphics library to the game library, a translation of the events is done.

All key codes are transformed to match SFML key macros according to the following codes:

- **Shortcuts:**

The core can detect some shortcuts like:

*Exit = q

*Go to menu = m ou M

✓ Graphicals libraries:

• General:

Graphicals libraries part deals with components to display them.
This part also take care of get event to send them to the game part.

• Interface:

```
class IDisplay {  
    public:  
        virtual ~IDisplay() = default;  
        virtual void init() = 0;  
        virtual bool isOpen() = 0;  
        virtual void display_menu() = 0;  
        virtual void close() = 0;  
        virtual std::string getActualGame() = 0;  
        virtual std::string getActualLibrary() = 0;  
        virtual void setActualGame(std::string) = 0;  
        virtual void setActualLibrary(std::string) = 0;  
        virtual void display_game_over() = 0;  
        virtual void display_score(int score) = 0;  
        virtual std::string handleEvents(std::string &direction) = 0;  
        virtual void display_state(const std::vector<std::vector<int>> &map) = 0;  
    protected:  
    private:  
};
```

Function explanations:

- Init: called when the display is launched
- isOpen: called when the window is open

- display_menu: function that allow us to display the menu of the games whatever librairies(ncurses, sfml, sdl)
- close: called when the display is close
- getActualGame: return the actual game
- getActualLibrary: return the actual library
- display_game_over: function called when the player lost the game
- display_score: function called when the game started to display the score of the player
- handle_events: function that return the events
- display_state: function that display the map of the game

✓ **Games:**

○ **General:**

The game is composed of two parts, the game logic / the actual game, and the part that says what should be displayed where and how so the display libs can display the game.

○ **Interface**

```

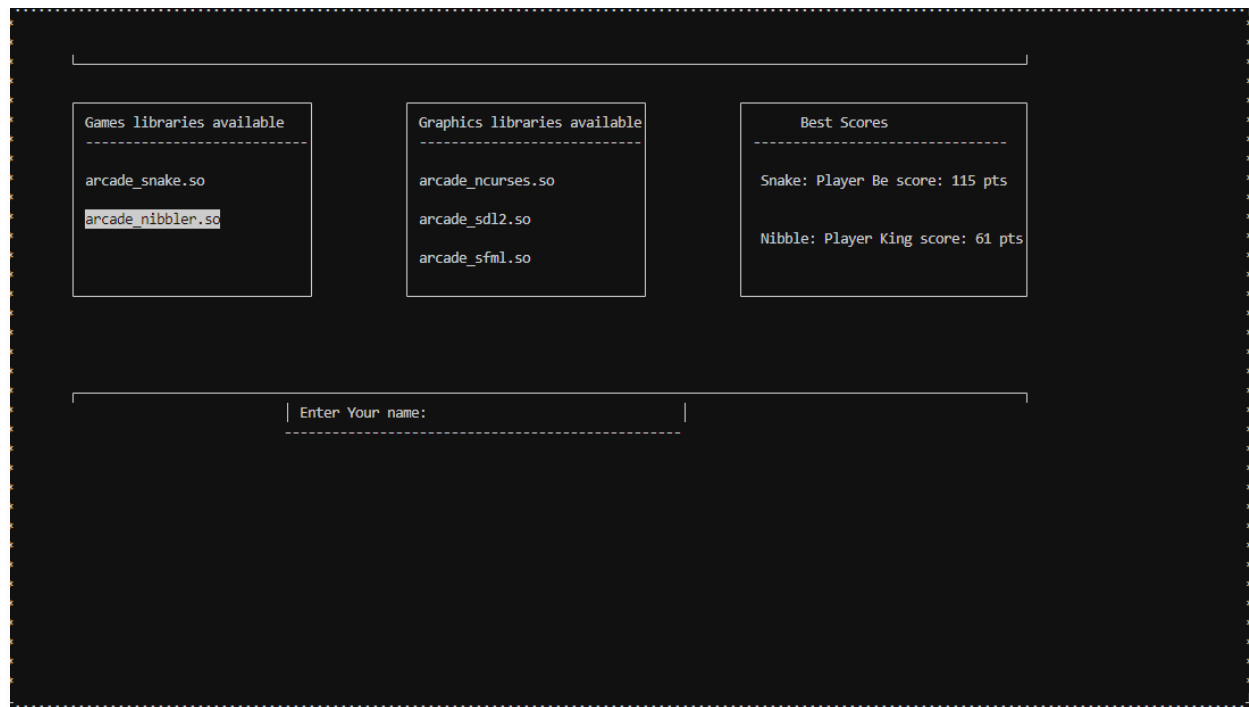
class IGames {
public:
    IGames() {}
    ~IGames() = default;
    virtual void init() = 0;
    virtual int getPlayerLive() = 0;
    virtual int getPlayerScore() = 0;
    virtual void update(std::string key) = 0;
    virtual std::vector<std::vector<int>> getState() = 0;
    virtual bool is_gameEnd() = 0;
    //virtual void setPlayerScore(int score) = 0;
    //virtual void saveState() = 0;
protected:
private:
};

```

- getPlayerlive: called to get the live of the player
- getPlayerScore: called to get the score of the player
- update: called to update the game with key
- getState: return state of the game
- is_gameEnd: return when the game is end

✓ Menu

The menu is a “fake” game where the point is to select the game and the display lib you want play and use, as well as show the highscores of other games. Choose your game and lib, set your name and start the game



Help

To change the player name, click on the current player name and start typing your name, once you have finished press enter to stop modifying the name. No game or lib will change until you click on the start button.

Commands

Select:

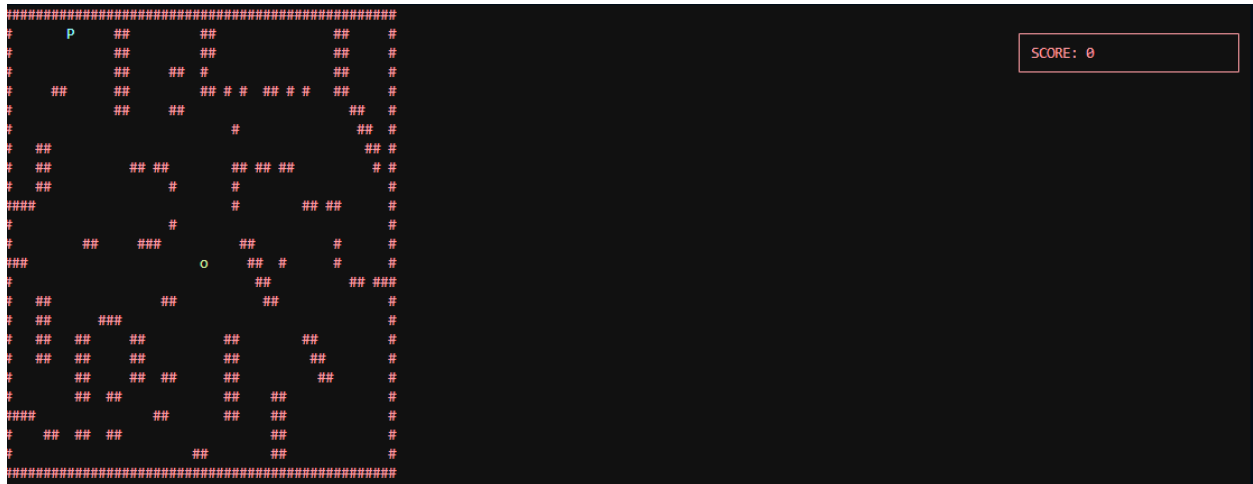
Mouse click: use your mouse to select the game and display lib you want

Else: M-> Go to menu

Nibbler

✓ Description

The Nibbler is a game where you are a snake trying to grow as much as possible, by eating apples.



✓ Game rules

The goal of the game is to have the highest score. You need to eat the maximum amount of apples you can, eating an apple gives you an extra point (+1) and makes the snake body grow. Hitting a wall, the border or even you own body with the snakes head makes you lose the game.

Snake

- Description

The goal of the game is to make the snake as long as possible. Various versions of Snake exist. Some of them include obstacles, others have a core system, or bonuses, etc. Sometimes, the map have no edge and has a cyclical play area where crossing through one boundary would have you appear on the opposite side.

✓ Games rules

1. Collision = Game over:
 - Hitting the walls (in classic mode) or the snake's own body ends the game
2. Food Mechanics:
 - New food spawns randomly when the snake eats the current one
3. Key controls
 - q->quit

Diagram class for our project

